OurMocks

2027 CYCLE

MOCK · PRACTICE · CRACK

CUET UG

Computer Science

Structured Query Language (SQL): DDL, DML & Functions

Data Definition & Manipulation Commands, Table Integrity Constraints, Query Filtering, and Built-in Scalar Functions

INSIDE
Concept Notes

INSIDE
Solved Examples

INSIDE
Practice MCQs + Solutions

Aligned to the latest official syllabus · Concept-first explanations

www.ourmocks.com

Syllabus Alignment

Section / Unit	Official NTA Syllabus Pointers	Core Focus & High-Yield Exam Traps
Section B1: Topic 6 (Class 12)	Structured Query Language: Introduction to SQL, DDL commands (CREATE, DROP, ALTER, TRUNCATE), data types (CHAR, VARCHAR, INT, FLOAT, DATE).	CHAR vs VARCHAR storage calculation, TRUNCATE (DDL, auto-commit) vs DELETE (DML, rollback), ALTER TABLE clauses (ADD, MODIFY, DROP).
Section B1: Topic 6 (Class 12)	Integrity Constraints & DML: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, DEFAULT, INSERT, UPDATE, DELETE commands.	UNIQUE permitting multiple NULLs vs PRIMARY KEY forbidding NULLs, catastrophic UPDATE/DELETE without WHERE, Referential actions.
Section B1: Topic 6 (Class 12)	DQL & Built-in Functions: SELECT, WHERE, ORDER BY, special operators (BETWEEN, IN, IS NULL, LIKE), single-row functions (String, Math, Date).	NULL comparison trap (`IS NULL` vs `= NULL`), 1-based indexing in `SUBSTR`, negative argument rounding in `ROUND(X, -D)`, WHERE vs HAVING.

EXAM PATTERN

SQL questions in CUET Computer Science focus extensively on subtle execution rules and edge cases:

1. **CHAR vs VARCHAR:** `CHAR(n)` is fixed-length and right-pads with spaces; `VARCHAR(n)` is variable-length and allocates only required characters plus length bytes.
2. **TRUNCATE vs DELETE vs DROP:** `DROP` deletes table schema + data; `TRUNCATE` deletes all rows instantly and resets storage (DDL, auto-commit); `DELETE` deletes rows with transaction logging (DML, can be rolled back).
3. **NULL Comparisons:** `NULL = NULL` evaluates to `UNKNOWN` (falsy in WHERE). Always use `IS NULL` or `IS NOT NULL`.
4. **SUBSTR() Indexing:** SQL strings are **1-indexed**, not 0-indexed! Negative start positions count backwards from string end.
5. **ROUND(X, D) with Negative D:** Rounds to the left of the decimal point (e.g. `ROUND(156.45, -1) = 160`, `ROUND(156.45, -2) = 200`).
6. **WHERE vs HAVING:** `WHERE` cannot contain aggregate functions (`COUNT`, `AVG`). Column aliases defined in `SELECT` cannot be referenced in `WHERE`.

Introduction to Structured Query Language (SQL)

Structured Query Language (SQL) is the standard declarative, set-oriented computer language utilized to define, access, manipulate, and administer relational databases. Developed initially by IBM researchers Donald Chamberlin and Raymond Boyce in the early 1970s (originally named SEQUEL for **Structured English Query Language**), SQL is standardized by ANSI (American National Standards Institute) and ISO (International Organization for Standardization):

Declarative Nature of SQL

Unlike traditional procedural languages (such as Python, C++, or Java) where the programmer must explicitly specify **how** to access data step-by-step (e.g., opening files, iterating pointers, traversing hash tables), SQL is strictly **declarative** (non-procedural):

- The user specifies **what** data is required by formulating relational predicates.
- The underlying RDBMS Query Optimizer autonomously determines **how** to retrieve it by constructing execution trees, choosing index lookups, and orchestrating join orders.

Classification of SQL Sublanguages

SQL commands are partitioned into four major functional sublanguages based on their target abstraction:

Sublanguage	Full Name	Primary Role & Scope	Key Command Keywords
DDL	Data Definition Language	Defines, alters, and drops database structures, tables, views, and integrity constraints. Implicitly auto-committed.	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`
DML	Data Manipulation Language	Manipulates data instances (tuples) within existing relations. Transactional; can be rolled back before commit.	`INSERT`, `UPDATE`, `DELETE`, `SELECT` (DQL)
DCL	Data Control Language	Administers access privileges, permissions, and security roles for database users.	`GRANT`, `REVOKE`
TCL	Transaction Control Language	Manages transaction boundaries, atomic commit states, and intermediate savepoints.	`COMMIT`, `ROLLBACK`, `SAVEPOINT`

SQL Data Types & Storage Semantics

Every attribute in an SQL relation must have a declared data type that defines its domain of permissible atomic values:

1. Character Types: CHAR(n) vs VARCHAR(n)

The distinction between fixed-length and variable-length text is one of the most heavily tested concepts in CUET:

- `CHAR(n)`: **Fixed-length** character string of size n (where $1 \leq n \leq 255$). If a stored string has fewer than n characters, the database automatically right-pads the value with blank spaces up to length n. Useful for standardized codes (e.g., `StateCode CHAR(2)`, `Gender CHAR(1)`).
- `VARCHAR(n)`: **Variable-length** character string up to a maximum size of n (where $1 \leq n \leq 65535$). The database allocates only the memory required to store the actual characters plus 1 or 2 prefix length bytes. No trailing spaces are padded. Ideal for names, addresses, and variable text.

> **CHAR VS VARCHAR STORAGE ALLOCATION INVARIANT**
>
> If the value `'Delhi'` (length 5) is stored into:
>
> - `City CHAR(10)`: Consumes strictly **10 bytes** of storage (stores `'Delhi '` with 5 trailing spaces).
> - `City VARCHAR(10)`: Consumes **6 bytes** of storage (5 bytes for characters + 1 byte for length prefix).

2. Numeric Data Types

- `INT` / `INTEGER`: Standard 4-byte signed integer spanning -2^{31} to $+2^{31} - 1$.
- `DECIMAL(P, D)` / `NUMERIC(P, D)`: Exact fixed-point numeric type. P represents total precision (total significant digits); D represents scale (digits to the right of the decimal point). Example: `DECIMAL(6, 2)` can store numbers up to 9999.99.
- `FLOAT` / `DOUBLE`: Approximate IEEE-754 floating-point numbers.

3. Temporal Data Types

- `DATE`: Represents calendar dates formatted as `'YYYY-MM-DD'` spanning `'1000-01-01'` to `'9999-12-31'`.
- `TIME`: Represents time of day formatted as `'HH:MM:SS'`.

- `DATETIME` / `TIMESTAMP`: Combined date and time formatted as `'YYYY-MM-DD HH:MM:SS'`.

Data Definition Language (DDL): Creating & Managing Schemas

1. The CREATE TABLE Command

The `CREATE TABLE` command constructs a new relation schema in the active database. Syntax structure:

```
CREATE TABLE TableName (
    Column1 DataType [ColumnConstraint],
    Column2 DataType [ColumnConstraint],
    ...,
    [TableConstraint(s)]
);
```

2. Integrity Constraints in Table Creation

Constraints enforce business rules and maintain relational consistency across all database operations:

- `NOT NULL`: Enforces that an attribute cannot be left empty or missing. Every inserted tuple must provide an explicit value.
- `UNIQUE`: Guarantees that all values in the column are distinct. Note: Unlike primary keys, a `UNIQUE` column **can accept multiple NULL values** under standard SQL (because each `NULL` is treated as distinct/unknown).
- `PRIMARY KEY`: Declares the unique entity identifier. It is the combination of `UNIQUE` and `NOT NULL`. A relation can have **at most one** Primary Key.
- `FOREIGN KEY`: Enforces referential integrity pointing to the Primary Key of a parent table. Child values must match an existing parent primary key value or be `NULL`.
- `CHECK`: Validates that attribute values satisfy a specified boolean predicate condition (e.g., `CHECK (Age >= 18)`).
- `DEFAULT`: Automatically assigns a specified fallback value when an attribute is omitted from an `INSERT` statement.

Column-Level vs Table-Level Constraint Definitions

SQL provides two distinct syntactic mechanisms to declare integrity constraints:

1. **Column-Level Constraints:** Declared inline immediately after the column definition. Column-level syntax can apply to **only one single column**:
   ```
   RollNo INT PRIMARY KEY,
   Email VARCHAR(100) UNIQUE NOT NULL
   ```
2. **Table-Level Constraints:** Declared as independent clauses at the end of the `CREATE TABLE` definition, separated by commas. Table-level syntax is **strictly required** when defining **composite keys** (keys comprising two or more columns) or naming constraints explicitly:
   ```
   CONSTRAINT PK_Enrollment PRIMARY KEY (RollNo, SubjectCode),
   CONSTRAINT FK_Student FOREIGN KEY (RollNo) REFERENCES STUDENT(RollNo)
   ```

Constraint Type	Column-Level Supported?	Table-Level Supported?
`PRIMARY KEY`	Yes (single-column primary keys only).	Yes (required for multi-column composite keys).
`UNIQUE`	Yes (single-column unique checks).	Yes (supports composite uniqueness over (A, B)).
`FOREIGN KEY`	Yes (abbreviated `REFERENCES Parent(PK)`).	Yes (full `FOREIGN KEY (...) REFERENCES ...`).
`CHECK`	Yes (checks individual column value).	Yes (can compare multiple columns, e.g. `EndDt > StartDt`).
`DEFAULT`	Yes (standard inline clause).	No (cannot be declared as a separate table-level clause).
`NOT NULL`	Yes (standard inline column modifier).	No (standard SQL does not support table-level NOT NULL).

COMPREHENSIVE DDL TABLE CREATION WITH FOREIGN KEY ACTIONS

Task: Write the SQL DDL statements to create a `DEPARTMENT` table and an `EMPLOYEE` table where deleting a department automatically sets employee `DeptId` to `NULL`.
Solution:

```
CREATE TABLE DEPARTMENT (
    DeptId INT PRIMARY KEY,
    DeptName VARCHAR(50) NOT NULL UNIQUE,
    Location VARCHAR(50) DEFAULT 'Headquarters'
);

CREATE TABLE EMPLOYEE (
```

```
    EmpId INT PRIMARY KEY,
    EmpName VARCHAR(60) NOT NULL,
    Salary DECIMAL(10, 2) CHECK (Salary > 0),
    Email VARCHAR(100) UNIQUE,
    DeptId INT,
    FOREIGN KEY (DeptId) REFERENCES DEPARTMENT(DeptId)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);
```

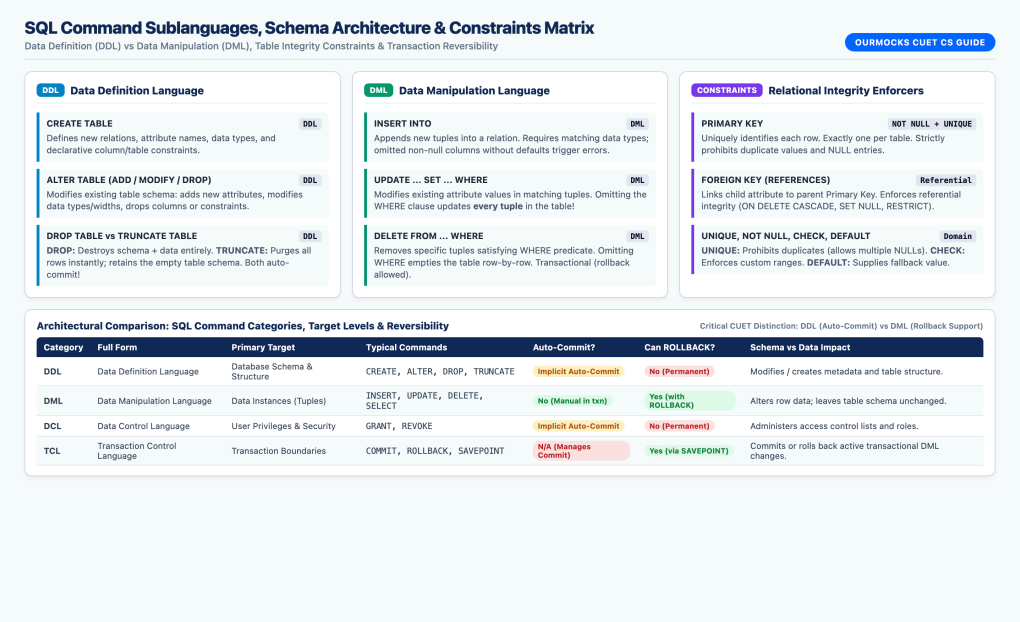


Figure 1: SQL Command Architecture: DDL vs DML, Integrity Constraints, and Transaction Reversibility Matrix

3. Modifying Schemas with ALTER TABLE

The `ALTER TABLE` command dynamically alters the structure of an existing relation without dropping its data:

1. **Adding a New Column:**
   `ALTER TABLE EMPLOYEE ADD (DateOfBirth DATE);`
2. **Modifying Existing Column Data Type or Width:**
   `ALTER TABLE EMPLOYEE MODIFY (EmpName VARCHAR(80) NOT NULL);`
   (Rule: Column width can generally be increased freely; decreasing width requires existing data to fit).
3. **Dropping an Existing Column:**
   `ALTER TABLE EMPLOYEE DROP COLUMN DateOfBirth;`
4. **Adding a Table Constraint:**
   `ALTER TABLE EMPLOYEE ADD CONSTRAINT PK_Emp PRIMARY KEY (EmpId);`
5. **Dropping a Constraint:**
   `ALTER TABLE EMPLOYEE DROP PRIMARY KEY;`

4. Removing Tables: DROP TABLE vs TRUNCATE TABLE

- `DROP TABLE TableName;` Completely destroys the entire relation definition from the database catalog along with all its tuples, indexes, and constraints. Once dropped, the table no longer exists.
- `TRUNCATE TABLE TableName;` Flushes all records from the table instantaneously by deallocating data pages, but **preserves the empty table schema and column definitions**. The table remains ready to accept new rows.
- **Critical Rule:** Both `DROP` and `TRUNCATE` are **DDL commands**. They issue an implicit commit and **cannot be rolled back**!

Data Manipulation Language (DML): Managing Tuples

DML commands operate exclusively on the rows (tuples) stored within existing relation schemas:

1. Inserting Data: INSERT INTO

The `INSERT INTO` statement appends one or more new tuples to a relation:

- **Explicit Column List:**

```
INSERT INTO EMPLOYEE (EmpId, EmpName, Salary, DeptId)
VALUES (101, 'Ananya', 75000.00, 10);
```

- **Implicit Column Order (All Columns):**
  `INSERT INTO EMPLOYEE VALUES (102, 'Rohan', 62000.00, 'rohan@corp.com', 20);`
- **Inserting Multiple Rows:**

```
INSERT INTO DEPARTMENT (DeptId, DeptName) VALUES
    (10, 'Engineering'),
    (20, 'Marketing'),
    (30, 'Finance');
```

2. Modifying Data: UPDATE ... SET

The `UPDATE` statement modifies existing attribute values in one or more tuples:

```
UPDATE EMPLOYEE
SET Salary = Salary * 1.10, Email = 'ananya_lead@corp.com'
WHERE DeptId = 10;
```

THE UNBOUNDED UPDATE / DELETE TRAP

If the `WHERE` clause is omitted from an `UPDATE` or `DELETE` statement:
`UPDATE EMPLOYEE SET Salary = 50000;`
The database will unconditionally update **every single tuple** in the relation! In production, this results in catastrophic data loss. Always verify the `WHERE` predicate before running update statements.

3. Removing Data: DELETE FROM

The `DELETE FROM` statement removes tuples that satisfy a boolean predicate:

```
DELETE FROM EMPLOYEE WHERE DeptId = 30 AND Salary < 40000;
```

- Omitting `WHERE` (`DELETE FROM EMPLOYEE;`) removes all tuples one by one, writing deletion logs for each tuple. Because it is a DML command, it can be reversed using `ROLLBACK` within an active transaction!

Data Query Language (DQL): The SELECT Statement

The `SELECT` statement is the primary mechanism for querying, filtering, and projecting relational data:

1. Fundamental Query Syntax

```
SELECT [DISTINCT] AttributeList
FROM TableName
[WHERE FilterCondition]
[ORDER BY SortColumn [ASC | DESC]];
```

2. Projecting All Columns vs Specific Columns

- `SELECT * FROM EMPLOYEE;` — Projects every attribute defined in the relation schema.
- `SELECT EmpName, Salary FROM EMPLOYEE;` — Projects only the specified attributes (Relational Algebra π).
- `SELECT DISTINCT DeptId FROM EMPLOYEE;` — Purges duplicate values from the output stream.

3. Filtering Tuples with the WHERE Clause

The `WHERE` clause performs horizontal selection (σ) based on boolean expressions using relational and logical operators:

Operator Category	Operator Symbols	Semantics & Behavior
Relational	`=`, `<>`, `!=`, `<`, `>`, `<=`, `>=`	Compares attribute values against literals or other attributes. Note: `<>` and `!=` both denote 'not equal to'.
Logical	`AND`, `OR`, `NOT`	Combines multiple conditions. Precedence order: `NOT` highest, then `AND`, then `OR`.
Range Search	`BETWEEN lower AND upper`	Tests if a value lies within an **inclusive** continuous range (lower $\leq$ value $\leq$ upper).
Set Membership	`IN (v1, v2, ...)`	Tests if a value matches any literal within a discrete parenthesized list.
Null Check	`IS NULL`, `IS NOT NULL`	Tests for missing or unknown values. **Never write `= NULL`!**
Pattern Match	`LIKE 'pattern'`	Performs string pattern matching using wildcards `%` (any string) and `_` (any single character).

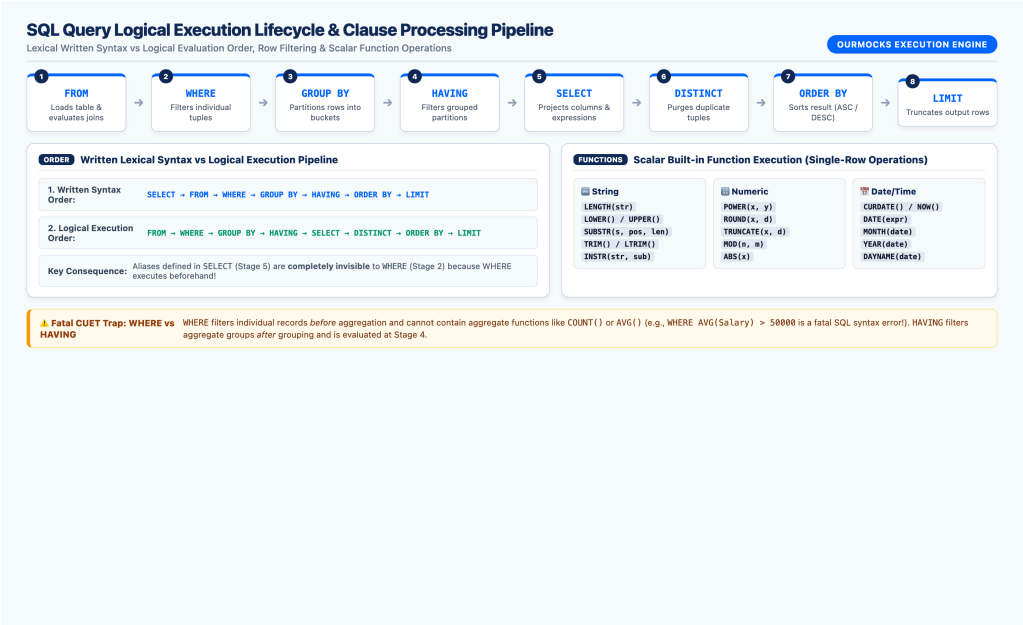


Figure 2: SQL Logical Query Execution Lifecycle: 8-Stage Clause Processing Pipeline & Scalar Evaluation

4. Three-Valued Logic and NULL Semantics

In relational theory, `NULL` does **not** represent zero (`0`) or an empty string (`''`). It signifies **missing, unknown, or inapplicable data**:

- **The Incomparability Invariant:** Any arithmetic comparison involving `NULL` evaluates to `UNKNOWN`:
  - `Salary = NULL` → `UNKNOWN` (Falsy in `WHERE`)
  - `Salary != NULL` → `UNKNOWN` (Falsy in `WHERE`)
  - `Salary + 1000` (where Salary is NULL) → `NULL`
- **Filtering Truth Value:** A `WHERE` clause retains a tuple if and only if the predicate evaluates to `TRUE`. Tuples evaluating to `FALSE` or `UNKNOWN` are strictly rejected.
- **Correct Syntax:** Always use `IS NULL` or `IS NOT NULL`:
  `SELECT EmpName FROM EMPLOYEE WHERE DeptId IS NULL;`

Three-Valued Logic (3VL) Truth Tables

SQL implements a three-valued logic system incorporating `TRUE`, `FALSE`, and `UNKNOWN`. Because `NULL` signifies missing data, logical operations combining `UNKNOWN` follow these exact truth tables:

Condition P	Condition Q	P AND Q	P OR Q
TRUE	TRUE	TRUE	TRUE
TRUE	FALSE	FALSE	TRUE
TRUE	UNKNOWN	UNKNOWN	TRUE
FALSE	FALSE	FALSE	FALSE
FALSE	UNKNOWN	FALSE	UNKNOWN
UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN

Condition P	NOT P
TRUE	FALSE
FALSE	TRUE
UNKNOWN	UNKNOWN

THE LAW OF EXCLUDED MIDDLE FAILS IN SQL

In classical two-valued logic, the predicate (P OR NOT P) is always guaranteed to evaluate to `TRUE`. In SQL, if an employee has `Commission IS NULL`, the condition:
`WHERE Commission > 500 OR Commission <= 500;`
evaluates to `UNKNOWN OR UNKNOWN = UNKNOWN`! Consequently, that employee tuple will be **silently omitted** from query results!

5. Pattern Matching with the LIKE Operator

The `LIKE` operator matches strings against search patterns using two standard wildcards:

1. **Percent Sign (`%`):** Matches any sequence of **zero or more** characters.
   - `LIKE 'A%'`: Matches any string starting with `'A'` (e.g. `'Alice'`, `'Amit'`, `'A'`).
   - `LIKE '%sh'`: Matches any string ending with `'sh'` (e.g. `'Suresh'`, `'Ramesh'`).
   - `LIKE '%tech%'`: Matches any string containing the substring `'tech'` anywhere.
2. **Underscore (`_`):** Matches **exactly one** arbitrary character.
   - `LIKE '_a%'`: Matches strings having `'a'` as their second character (e.g. `'Raj'`, `'Karan'`).
   - `LIKE '___'`: Matches strings of **exactly 3 characters** in length.
   - `LIKE '_ _ _%'`: Matches strings having at least 3 characters.

Escaping Wildcard Characters in LIKE Queries

If a database column contains literal percent signs (`%`) or underscores (`_`) (e.g. `'Discount 20%'` or `'Item_A'`), standard wildcard interpretation must be bypassed using the `ESCAPE` clause:

```
SELECT * FROM PRODUCT WHERE ProductName LIKE '%20!%%' ESCAPE '!';
```

In this query, the exclamation mark `!` is designated as the escape character. The sequence `!%` is treated as a literal percent sign, while the outer `%` characters act as regular wildcards.

FILTERING QUERIES WITH SPECIAL OPERATORS

Problem: Consider table `STUDENT(RollNo, Name, Marks, Grade)`:

1. Write a query to find all students whose marks are between 80 and 90 (inclusive).
2. Write a query to find students whose Name has `'e'` as the third character.
3. Write a query to find students whose Grade is neither `'A'` nor `'B'`.

Solution:

- **Part 1:**
  ```
  SELECT * FROM STUDENT WHERE Marks BETWEEN 80 AND 90;
  ```
  (Equivalent to: `WHERE Marks >= 80 AND Marks <= 90`)
- **Part 2:**
  ```
  SELECT * FROM STUDENT WHERE Name LIKE '__e%';
  ```
  (Two underscores match the first two characters; `'e'` matches the 3rd; `%` matches any remaining characters).
- **Part 3:**
  ```
  SELECT * FROM STUDENT WHERE Grade NOT IN ('A', 'B');
  ```

Sorting Data: The ORDER BY Clause

The `ORDER BY` clause sorts output tuples in ascending or descending sequence according to one or more specified attributes:

- `ASC`: Ascending order (lowest to highest, A to Z). **This is the default ordering** if omitted.
- `DESC`: Descending order (highest to lowest, Z to A).

Multi-Column Ordering

When multiple columns are specified, sorting proceeds hierarchically from left to right:

```
SELECT EmpName, DeptId, Salary
FROM EMPLOYEE
ORDER BY DeptId ASC, Salary DESC;
```

Execution Trace: The engine sorts all rows primarily by `DeptId` in ascending order. For tuples sharing the identical `DeptId`, it sorts secondary order by `Salary` in descending order.

Sorting Behavior of NULL Values

Under ANSI standard SQL:

- In ascending sorts (`ASC`), `NULL` values appear either first or last depending on database engine defaults (in MySQL, `NULL`s sort **first** in `ASC`, and **last** in `DESC`).

SQL Built-in Scalar (Single-Row) Functions

Scalar Functions operate on individual input attribute values and return exactly **one result value for each input row**. They are categorized into String, Numeric, and Date/Time functions:

1. String Functions

Function Syntax	Example Query	Returned Output & Semantics
`LENGTH(str)`	`LENGTH('CUET UG')`	`7` (Returns length in bytes/characters).
`LOWER(str)` / `LCASE(str)`	`LOWER('COMPUTER')`	`'computer'` (Converts all characters to lowercase).
`UPPER(str)` / `UCASE(str)`	`UPPER('science')`	`'SCIENCE'` (Converts all characters to uppercase).
`SUBSTR(s, pos, len)` / `MID()`	`SUBSTR('DATABASE', 5, 4)`	`'BASE'` (Extracts 4 characters starting at 1-based index 5).
`SUBSTR(s, pos)`	`SUBSTR('INFORMATICS', 3)`	`'FORMATICS'` (Extracts from position 3 to end of string).
`SUBSTR(s, -pos, len)`	`SUBSTR('KNOWLEDGE', -4, 3)`	`'EDG'` (Negative position counts from string end: `'E'(-4),'D'(-3),'G'(-2),'E'(-1)`).
`INSTR(str, substr)`	`INSTR('PROGRAMMING', 'GRAM')`	`4` (Returns 1-based index of first occurrence; returns 0 if not found).
`TRIM(str)`	`TRIM('  SQL  ')`	`'SQL'` (Strips both leading and trailing whitespace).
`LTRIM(str)` / `RTRIM(str)`	`LTRIM('  CODE')`	`'CODE'` (`LTRIM` strips leading; `RTRIM` strips trailing).
`CONCAT(s1, s2, ...)`	`CONCAT('CUET', '-', '2027')`	`'CUET-2027'` (Concatenates strings; returns NULL if any argument is NULL).
`CONCAT_WS(sep, s1, s2, ...)`	`CONCAT_WS('-', 'CUET', 'UG', '2027')`	`'CUET-UG-2027'` (Concatenates with separator; gracefully skips NULLs).
`REPLACE(str, from, to)`	`REPLACE('DATA BASE', ' ', '')`	`'DATABASE'` (Replaces all occurrences of search string with replacement).
`LPAD(str, len, pad)`	`LPAD('CS', 5, '0')`	`'000CS'` (Left-pads string to designated length using pad string).
`RPAD(str, len, pad)`	`RPAD('75', 5, '*')`	`'75***'` (Right-pads string to designated length using pad string).

2. Numeric Functions

Function Syntax	Example Query	Returned Output & Semantics
`POWER(X, Y)` / `POW()`	`POWER(3, 4)`	`81.0` (Calculates X^Y, $3^4 = 81$).
`ROUND(X, D)`	`ROUND(174.568, 2)`	`174.57` (Rounds X to D decimal places; looks at $(D+1)$ -th digit).
`ROUND(X, 0)` / `ROUND(X)`	`ROUND(174.568)`	`175` (Rounds to nearest whole integer).
`ROUND(X, -1)`	`ROUND(174.568, -1)`	`170` (Negative D rounds to tens: checks units digit $4 < 5 \rightarrow 170$).
`ROUND(X, -2)`	`ROUND(174.568, -2)`	`200` (Rounds to hundreds: checks tens digit $7 \geq 5 \rightarrow 200$).
`TRUNCATE(X, D)`	`TRUNCATE(174.568, 2)`	`174.56` (Simply chops off digits beyond D without rounding!).
`TRUNCATE(X, -1)`	`TRUNCATE(174.568, -1)`	`170` (Replaces units place with zero without rounding).
`MOD(N, M)`	`MOD(29, 6)`	`5` (Returns remainder: $29 - 6 \times \lfloor \frac{29}{6} \rfloor = 5$).
`CEIL(X)` / `CEILING()`	`CEIL(14.15)`	`15` (Returns smallest integer greater than or equal to X).
`FLOOR(X)`	`FLOOR(14.85)`	`14` (Returns largest integer less than or equal to X).
`SIGN(X)`	`SIGN(-42.5)`	`-1` (Returns -1 for negative, 0 for zero, $+1$ for positive).
`SQRT(X)`	`SQRT(144)`	`12.0` (Returns square root; returns NULL for negative arguments).
`ABS(X)`	`ABS(-42.75)`	`42.75` (Returns absolute non-negative magnitude).

3. Date & Time Functions

Function Syntax	Example Query	Returned Output & Semantics
`CURDATE()` / `CURRENT_DATE`	`CURDATE()`	`'2027-05-15'` (Returns current date in `'YYYY-MM-DD'` format).
`NOW()` / `CURRENT_TIMESTAMP`	`NOW()`	`'2027-05-15 10:30:45'` (Returns current date and time).
`SYSDATE()`	`SYSDATE()`	Returns exact time at which the function executes (unlike `NOW()`).
`DATE(expr)`	`DATE('2027-05-15 10:30:45')`	`'2027-05-15'` (Extracts date part from a datetime expression).
`MONTH(date)`	`MONTH('2027-05-15')`	`5` (Returns numeric month component from 1 to 12).
`MONTHNAME(date)`	`MONTHNAME('2027-05-15')`	`'May'` (Returns full textual month name).
`YEAR(date)`	`YEAR('2027-05-15')`	`2027` (Returns four-digit year).
`DAYOFMONTH(date)` / `DAY()`	`DAYOFMONTH('2027-05-15')`	`15` (Returns day of the month from 1 to 31).
`DAYOFWEEK(date)`	`DAYOFWEEK('2027-05-15')`	`7` (ODBC standard: 1=Sun, 2=Mon, 3=Tue, 4=Wed, 5=Thu, 6=Fri, 7=Sat).
`DAYOFYEAR(date)`	`DAYOFYEAR('2027-02-01')`	`32` (Day of the year from 1 to 366).
`QUARTER(date)`	`QUARTER('2027-05-15')`	`2` (Returns calendar quarter 1 to 4).
`DATEDIFF(d1, d2)`	`DATEDIFF('2027-05-25', '2027-05-15')`	`10` (Calculates d1 − d2 in integer days).
`DAYNAME(date)`	`DAYNAME('2027-05-15')`	`'Saturday'` (Returns textual weekday name: Monday, Tuesday, etc.).

Advanced Query Composition & Function Nesting

In enterprise RDBMS environments and CUET examination problems, scalar functions are frequently nested to perform multi-stage data transformations within a single query expression:

EVALUATING COMPLEX NESTED STRING & DATE EXPRESSIONS

Problem: Consider the relation `STUDENT_ADMISSION`:

- `AdmNo: 'A-2027-094'`
- `FullName: '  rajesh sharma  '`
- `DOB: '2008-11-23'`

Evaluate the exact string returned by the following SQL query:

```
SELECT CONCAT(
    UPPER(TRIM(FullName)),
    ' (Born on ',
    DAYNAME(DOB),
    ', Year ',
    YEAR(DOB),
    ')'
) AS StudentBio;
```

Solution:

1. `TRIM(FullName)` strips surrounding whitespace → `'rajesh sharma'`.
2. `UPPER('rajesh sharma')` converts all characters to uppercase → `'RAJESH SHARMA'`.
3. `DAYNAME('2008-11-23')`: November 23, 2008 fell on a Sunday → `'Sunday'`.
4. `YEAR('2008-11-23')` extracts the 4-digit year → `2008`.
5. `CONCAT` concatenates the evaluated arguments:
   - `'RAJESH SHARMA'`
   - `' (Born on '`
   - `'Sunday'`
   - `', Year '`
   - `'2008'`
   - `')'`
6. **Final Output:** `'RAJESH SHARMA (Born on Sunday, Year 2008)'`.

TRANSACTIONAL EXECUTION TRACING WITH SAVEPOINT AND ROLLBACK

Problem: A database session executes the following sequential commands on table `ACCOUNT(AccNo, Balance)` containing tuple `(101, 5000)`:

```
START TRANSACTION;
UPDATE ACCOUNT SET Balance = Balance + 2000 WHERE AccNo = 101;
SAVEPOINT sp1;
UPDATE ACCOUNT SET Balance = Balance - 1500 WHERE AccNo = 101;
SAVEPOINT sp2;
DELETE FROM ACCOUNT WHERE AccNo = 101;
ROLLBACK TO SAVEPOINT sp1;
COMMIT;
```

What is the final state and balance of account `101` in the database?

Solution:

1. `UPDATE Balance = Balance + 2000`: Balance becomes $5000 + 2000 = 7000$.
2. `SAVEPOINT sp1`: Snapshot created at Balance $= 7000$.
3. `UPDATE Balance = Balance - 1500`: Balance becomes $7000 - 1500 = 5500$.
4. `SAVEPOINT sp2`: Snapshot created at Balance $= 5500$.
5. `DELETE WHERE AccNo = 101`: Tuple is marked deleted.
6. `ROLLBACK TO SAVEPOINT sp1`: Reverts all actions performed after `sp1`. The `DELETE` and the second `UPDATE` are undone! The state at `sp1` is restored (Balance $= 7000$).
7. `COMMIT`: Persists the state at `sp1` permanently to disk.
8. **Final Result:** Account `101` exists with Balance $= 7000$.

EVALUATING COMPLEX SCALAR FUNCTION EXPRESSIONS

Problem: Evaluate the exact output returned by each of the following SQL expressions:

1. `SELECT SUBSTR('CENTRAL BOARD', INSTR('CENTRAL BOARD', 'BO'), 5);`
2. `SELECT ROUND(287.64, -2) + TRUNCATE(45.98, 1);`
3. `SELECT MOD(ROUND(15.7), 4);`

Solution:

- **Part 1:**
  - `INSTR('CENTRAL BOARD', 'BO')` searches for substring `'BO'`. Found at 1-based index 9 (`'CENTRAL '` has length 8, so `'B'` is at 9).
  - `SUBSTR('CENTRAL BOARD', 9, 5)` extracts 5 characters starting at position 9. Characters at positions 9, 10, 11, 12, 13 are `'B'`, `'O'`, `'A'`, `'R'`, `'D'`.
  - **Output:** `'BOARD'`
- **Part 2:**
  - `ROUND(287.64, -2)`: Negative 2 rounds to the nearest hundreds place. Tens digit is $8 \geq 5$, so it rounds up to 300.
  - `TRUNCATE(45.98, 1)`: Simply truncates to 1 decimal place without rounding, giving 45.9.
  - Sum: $300 + 45.9 = 345.9$.
  - **Output:** `345.9`
- **Part 3:**
  - `ROUND(15.7)` rounds to nearest whole number $= 16$.
  - `MOD(16, 4) = 16 mod 4 = 0`.
  - **Output:** `0`

Master Synthesis Mindmap: Structured Query Language (SQL) Basics

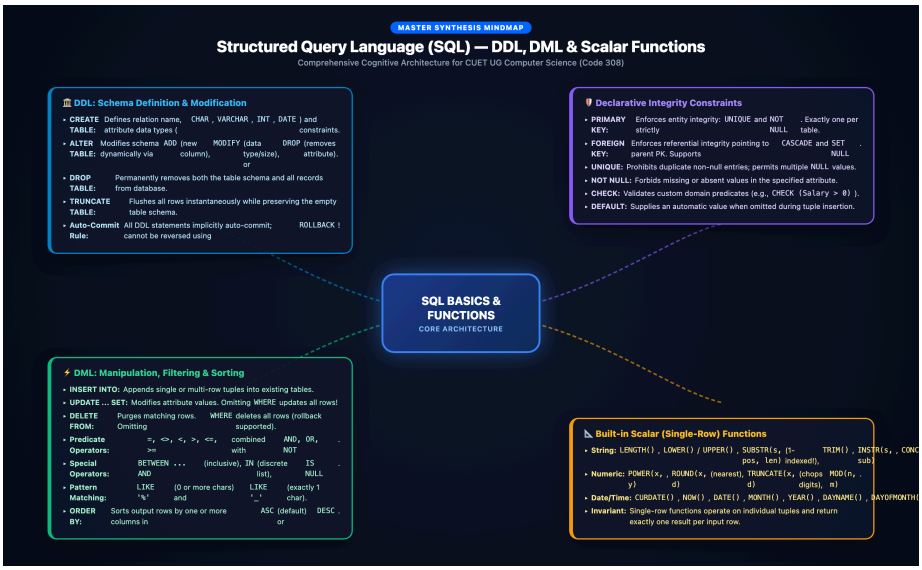


Figure 3: Master Synthesis Mindmap: SQL Commands, DDL & DML Architecture, Integrity Constraints & Scalar Functions

OurMocks

Cheatsheet & The Seven Fatal CUET Traps

RAPID REVISION CHEATSHEET: STRUCTURED QUERY LANGUAGE (SQL) BASICS

- **SQL Sublanguages:** DDL (Schema, auto-commit, `CREATE/ALTER/DROP/TRUNCATE`), DML (Data, rollback supported, `INSERT/UPDATE/DELETE`), DCL (`GRANT/REVOKE`), TCL (`COMMIT/ROLLBACK/SAVEPOINT`).
- **CHAR vs VARCHAR:** `CHAR(n)` fixed-length, right-pads spaces; `VARCHAR(n)` variable-length, stores actual length + 1/2 bytes.
- **Constraints:** `PRIMARY KEY` (Unique + NOT NULL, 1 per table); `FOREIGN KEY` (links child to parent PK; `CASCADE`, `SET NULL`, `RESTRICT`); `UNIQUE` (no duplicate non-nulls; allows multiple NULLs); `CHECK` (range/domain validation).
- **ALTER TABLE Syntax:** `ADD (col DataType)` adds column; `MODIFY (col DataType)` alters type/size; `DROP COLUMN col` deletes column.
- **DML Traps:** `UPDATE table SET ...` and `DELETE FROM table;` without `WHERE` affect **every tuple** in the relation!
- **NULL Mechanics:** `NULL` denotes missing/unknown. `NULL = NULL` is `UNKNOWN`. Always use `IS NULL` or `IS NOT NULL`.
- **Pattern Matching:** `LIKE '%'` matches 0 or more chars; `LIKE '_'` matches exactly 1 char. Case-insensitive in MySQL.
- **ORDER BY:** Default is `ASC`. Multi-column `ORDER BY C1 ASC, C2 DESC` sorts by C1 first; ties resolved by C2.
- **String Functions:** SQL is **1-indexed**! `SUBSTR(s, pos, len)`, `INSTR(s, sub)` returns 0 if not found, `TRIM(s)`, `LENGTH(s)`.
- **Numeric Functions:** `ROUND(X, D)` (rounds, checks $(D+1)$-th digit); `TRUNCATE(X, D)` (chops digits, no rounding); `ROUND(X, -1)` rounds to tens! `MOD(N, M) = N - M * floor(N/M)`.
- **Date Functions:** `CURDATE()` = `'YYYY-MM-DD'`, `NOW()` = `'YYYY-MM-DD HH:MM:SS'`, `MONTH()`, `YEAR()`, `DAYNAME()`.

THE SEVEN FATAL CUET TRAPS IN SQL BASICS

1. **The 1-Based String Indexing Trap:** In SQL, strings start at index **1**, not index **0**! `SUBSTR('COMPUTER', 1, 3)` yields `'COM'`. If you specify index 0, MySQL returns an empty string or standard warning.
2. **The Negative Argument Rounding Trap:** When $D < 0$ in `ROUND(X, D)`, rounding occurs to the **left** of the decimal point: `ROUND(473.82, -1) = 470`, `ROUND(473.82, -2) = 500`. Do not confuse with decimal truncation!
3. **The TRUNCATE vs DELETE Rollback Trap:** `DELETE FROM EMP;` is DML (logs rows, can be undone with `ROLLBACK`). `TRUNCATE TABLE EMP;` is DDL (deallocates pages, issues implicit auto-commit, cannot be rolled back!).
4. **The NULL Equality Operator Trap:** Writing `WHERE Salary = NULL` or `WHERE Salary != NULL` produces syntax that runs but returns **zero rows** because comparison with `NULL` yields `UNKNOWN`. Always write `WHERE Salary IS NULL`.
5. **The BETWEEN Inclusive Range Trap:** `BETWEEN 10 AND 20` is strictly **inclusive** ($10 \leq X \leq 20$). If an option claims it is exclusive ($10 < X < 20$), it is an incorrect distractor!
6. **The WHERE Clause Alias Trap:** Column aliases defined in `SELECT` (e.g., `SELECT Salary * 12 AS AnnualPay`) **cannot** be referenced in the `WHERE` clause (`WHERE AnnualPay > 500000` causes an error) because `WHERE` executes **before** `SELECT`!
7. **The UNIQUE Constraint Multiple NULLs Trap:** A `UNIQUE` constraint permits multiple `NULL` values under ANSI standard SQL because two `NULL`s are not considered equal to each other.

CUET Focused Multiple Choice Questions (Part 1)

Q1. A database administrator stores the string `'INDIA'` in two columns defined respectively as `Country1 CHAR(10)` and `Country2 VARCHAR(10)`. Which of the following statements correctly describes the storage allocation in bytes for these two columns?

(A) `Country1` consumes 10 bytes (padded with spaces); `Country2` consumes 6 bytes (5 data bytes + 1 length byte)
(B) `Country1` consumes 5 bytes; `Country2` consumes 10 bytes
(C) Both columns consume identical storage of 5 bytes
(D) Both columns consume identical storage of 10 bytes

Sol. Q1. `CHAR(n)` is fixed-length; storing a 5-character string in `CHAR(10)` allocates the full 10 bytes by right-padding 5 spaces. `VARCHAR(n)` is variable-length; it allocates only the actual character length (5 bytes) plus a 1-byte length prefix, totaling 6 bytes.
Answer: (A)

Q2. Which of the following SQL statements belongs strictly to the Data Definition Language (DDL) and cannot be undone using a `ROLLBACK` command?

(A) `DELETE FROM STUDENT;`
(B) `TRUNCATE TABLE STUDENT;`
(C) `UPDATE STUDENT SET Marks = 0;`
(D) `INSERT INTO STUDENT VALUES (101, 'Ananya');`

Sol. Q2. `TRUNCATE TABLE` is a DDL command that deallocates entire data pages and issues an implicit auto-commit. It cannot be rolled back. In contrast, `DELETE`, `UPDATE`, and `INSERT` are DML commands that log row-level modifications and can be rolled back within an active transaction.
Answer: (B)

Q3. Consider the relation `STUDENT(RollNo INT, Name VARCHAR(50), Email VARCHAR(50) UNIQUE)`. If three separate insert statements insert tuples with `Email = NULL`, what is the outcome under standard SQL?

(A) Only the first tuple is inserted; the second and third trigger a UNIQUE constraint violation
(B) A runtime compile error occurs because UNIQUE strictly forbids NULL
(C) All three tuples are successfully inserted because UNIQUE allows multiple NULL values
(D) All NULL values are automatically converted into empty strings `''`

Sol. Q3. Under ANSI standard SQL, the `UNIQUE` constraint prohibits duplicate non-null values. Because `NULL` represents an unknown value and `NULL = NULL` is not true, multiple rows can contain `NULL` in a `UNIQUE` column. (Only `PRIMARY KEY` forbids `NULL` completely).
Answer: (C)

Q4. What is the exact result of evaluating the SQL expression `SELECT ROUND(384.652, -2);`?

(A) `384.65`
(B) `380.00`
(C) `300`
(D) `400`

Sol. Q4. A negative argument -2 in `ROUND(X, -2)` instructs the engine to round to the nearest hundreds place. We inspect the tens place digit (8). Because $8 \geq 5$, the number rounds up to 400.
Answer: (D)

Q5. Which of the following queries correctly identifies all employees whose `DeptId` is missing or unassigned?

(A) `SELECT * FROM EMPLOYEE WHERE DeptId IS NULL;`
(B) `SELECT * FROM EMPLOYEE WHERE DeptId = NULL;`
(C) `SELECT * FROM EMPLOYEE WHERE DeptId == NULL;`
(D) `SELECT * FROM EMPLOYEE WHERE DeptId IN (NULL);`

Sol. Q5. In SQL, `NULL` represents unknown data and cannot be compared using equality operators (`=` or `!=`), as such comparisons evaluate to `UNKNOWN`. To filter for `NULL` values, the special operator `IS NULL` must be utilized.
Answer: (A)

CUET Focused Multiple Choice Questions (Part 2)

Q6. A user executes the query: `SELECT SUBSTR('INFORMATICS PRACTICES', 3, 7);`. What is the string returned by the database?

(A) `'FORMATI'` **(B)** `'FORMAT'` (length 6)
(C) `'FORMATI'` (length 7, starting at 1-based index 3) **(D)** `'NFORMAT'`

Sol. Q6. SQL string indexing is **1-based**. In `'INFORMATICS PRACTICES'`, index 1 is `'I'`, index 2 is `'N'`, and index 3 is `'F'`. Extracting 7 characters starting from position 3 yields `'F', 'O', 'R', 'M', 'A', 'T', 'I'`.
Answer: (B)

Q7. Which command dynamically adds a new column `DateOfJoining DATE` with a default value of current date to an existing table `TEACHER`?

(A) `UPDATE TABLE TEACHER ADD (DateOfJoining DATE DEFAULT CURDATE());`
(B) `ALTER TABLE TEACHER INSERT COLUMN DateOfJoining DATE;`
(C) `ALTER TABLE TEACHER ADD (DateOfJoining DATE DEFAULT (CURDATE()));`
(D) `MODIFY TABLE TEACHER ADD DateOfJoining DATE;`

Sol. Q7. Modifying the structure of an existing table requires the `ALTER TABLE` command paired with the `ADD` clause. `UPDATE` is DML for modifying row data, and `MODIFY TABLE` is invalid SQL syntax.
Answer: (C)

Q8. What is the output of the query: `SELECT INSTR('EXAMINATION', 'MIN');`?

(A) `3` **(B)** `5`
(C) `0` **(D)** `4`

Sol. Q8. `INSTR(str, substr)` returns the 1-based starting position of the first occurrence of `substr` in `str`. In `'EXAMINATION'`: 'E'(1), 'X'(2), 'A'(3), 'M'(4), 'I'(5), 'N'(6). The substring `'MIN'` begins at position 4.
Answer: (D)

Q9. Consider the query: `SELECT EmpName FROM EMPLOYEE WHERE Salary BETWEEN 30000 AND 50000;`. Which of the following expressions is mathematically and logically equivalent?

(A) `WHERE Salary >= 30000 AND Salary <= 50000` **(B)** `WHERE Salary > 30000 AND Salary < 50000`
(C) `WHERE Salary >= 30000 OR Salary <= 50000` **(D)** `WHERE Salary IN (30000, 50000)`

Sol. Q9. The `BETWEEN ... AND` operator in SQL defines an **inclusive** range. Therefore, `Salary BETWEEN 30000 AND 50000` is strictly equivalent to `Salary >= 30000 AND Salary <= 50000`.
Answer: (A)

Q10. What is the output of evaluating the SQL expression `SELECT TRUNCATE(482.769, 1);`?

(A) `482.7` **(B)** `482.8`
(C) `480.0` **(D)** `482.77`

Sol. Q10. `TRUNCATE(X, D)` cuts off the number to D decimal places without performing any rounding whatsoever. While `ROUND(482.769, 1)` would round up to `482.8` because the next digit is 6, `TRUNCATE` simply discards digits beyond 1 decimal place, returning `482.7`.
Answer: (B)

CUET Focused Multiple Choice Questions (Part 3)

Q11. Which pattern in a `LIKE` clause matches any student name that contains exactly four characters and begins with the letter `'S'`?

(A) `LIKE 'S%'` **(B)** `LIKE 'S___%'`
(C) `LIKE 'S___'` **(D)** `LIKE 'S_ _ _ _'`

Sol. Q11. The underscore wildcard `_` matches exactly one arbitrary character. To match a name of exactly four characters beginning with `'S'`, we need the letter `'S'` followed by exactly three underscores (`'S___'`).
Answer: (C)

Q12. Why does the query `SELECT Salary * 12 AS AnnualSalary FROM EMP WHERE AnnualSalary > 600000;` fail with an error in standard SQL?

(A) Arithmetic multiplication cannot be performed inside the `SELECT` clause **(B)** The keyword `AS` is mandatory in `WHERE` clauses
(C) The `AnnualSalary` alias is created in `WHERE` before `SELECT` executes **(D)** The `WHERE` clause is logically executed before the `SELECT` clause, making `AnnualSalary` unknown

Sol. Q12. In SQL's logical execution pipeline, the `FROM` and `WHERE` clauses are evaluated at Stages 1 and 2, whereas `SELECT` (which defines column aliases) is evaluated at Stage 5. Consequently, aliases created in `SELECT` are completely invisible to the `WHERE` clause.
Answer: (D)

Q13. What is the result of the query `SELECT MOD(23, -5);` under standard MySQL/SQL arithmetic?

(A) `3` **(B)** `-3`
(C) `4` **(D)** `-4`

Sol. Q13. In SQL, `MOD(N, M)` is mathematically defined by the formula $N - M \times \text{floor}\left(\frac{N}{M}\right)$ or retains the sign of the numerator N. For `MOD(23, -5)`, $\frac{23}{-5} = -4.6 \to \text{floor} = -5$. Then $23 - (-5 \times -5) = 23 - 25 = -2$, but under MySQL integer remainder, $23 = (-5 \times -4) + 3$, so it returns $+3$. (The sign matches the dividend $N = 23$).
Answer: (A)

Q14. In an `ORDER BY` clause sorting multiple columns: `ORDER BY Stream ASC, Marks DESC`, how does the database resolve ties?

(A) It issues a duplicate key warning and truncates the tie **(B)** Tuples with identical `Stream` values are sorted among themselves by `Marks` in descending order
(C) It ignores `Stream` and sorts all records solely by `Marks` **(D)** The database sorts randomly whenever identical values appear

Sol. Q14. Multi-column sorting evaluates criteria from left to right. The primary sort orders all tuples by `Stream` ascending. Whenever two or more records possess identical `Stream` values, the tie is broken by sorting those specific records by `Marks` in descending order.
Answer: (B)

Q15. Which date function returns the current date in `'YYYY-MM-DD'` format without any time component?

(A) `NOW()` **(B)** `SYSDATE()`
(C) `CURDATE()` **(D)** `DATE_ADD()`

Sol. Q15. `CURDATE()` (or `CURRENT_DATE`) returns the current date formatted as `'YYYY-MM-DD'` without time. `NOW()` and `SYSDATE()` both return both date and time components (`'YYYY-MM-DD HH:MM:SS'`).
Answer: (C)